Table 1

ID	Description	Sensitive default
actuator	Provides a hypermedia-based 'discovery page' for the other endpoints. Requires Spring HATEOAS to be on the classpath.	True
auditevents	Exposes audit events information for the current application.	True
autoconfig	Displays an auto-configuration report showing all auto-configuration candidates and the reason why they 'were' or 'were not' applied.	True
beans	Displays a complete list of all the Spring Beans in your application.	True
configprops	Displays a collated list of all *@ConfigurationProperties*.	True
dump	Performs a thread dump.	True
env	Exposes properties from Spring's *ConfigurableEnvironment*.	True
flyway	Shows any Flyway database migrations that have been applied.	True
health	Shows application health information (when the application is secure, a simple 'status' when accessed over an unauthenticated connection or full message details when authenticated).	False
info	Displays arbitrary application information.	False
loggers	Shows and modifies the configuration of loggers in the application.	True
liquibase	Shows any Liquibase database migrations that have been applied.	True
metrics	Shows 'metrics' information for the current application.	True
mappings	Displays a collated list of all *@RequestMapping* paths.	True
shutdown	Allows the application to be gracefully shut down (not enabled by default).	True
trace	Displays trace information (by default, the last 100 HTTP requests).	True

Add the Spring Boot Admin Server configuration by adding *@EnableAdminServer* to your configuration, as follows:

```
package org.samrttechie;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.
SpringBootApplication;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.
builders.HttpSecurity;
import org.springframework.security.config.annotation.web.
configuration.WebSecurityConfigurerAdapter;

import de.codecentric.boot.admin.config.EnableAdminServer;

@EnableAdminServer
@Configuration
@SpringBootApplication
public class SpringBootAdminApplication {

	public static void main(String[] args) {
		SpringApplication.
run(SpringBootAdminApplication.class, args);
	}

	@Configuration
	public static class SecurityConfig extends
WebSecurityConfigurerAdapter {
		@Override
		protected void configure(HttpSecurity http)
throws Exception {
	// Page with login form is served as /login.html and does
a POST on /login
			http.formLogin().loginPage("/login.
html").loginProcessingUrl("/login").permitAll();
		// The UI does a POST on /logout on logout
			http.logout().logoutUrl("/logout");
		// The ui currently doesn't support csrf
			http.csrf().disable();

		// Requests for the login page and the static
assets are allowed
			http.authorizeRequests()
			.antMatchers("/login.html", "/**/*.
css", "/img/**", "/third-party/**")
			.permitAll();
		// ... and any other request needs to be authorized
			http.authorizeRequests().
antMatchers("/**").authenticated();

			// Enable so that the clients can
authenticate via HTTP basic for registering
			http.httpBasic();
		}
	}
	// end::configuration-spring-security[]

}
```

Let us create more Spring Boot applications to monitor through the Spring Boot Admin Server created in the above